



KTUNOTES.LIVE

KTU 2024 SCHEME NOTES

100% FREE RESOURCES

YOU CAN ACCESS B.TECH NOTES ON OUR THREE WEBSITES.

KTU2024.WEB.APP



SCAN ME TO GET THE WEBSITE

KTUNOTES.LIVE



SCAN ME TO GET THE WEBSITE

KTU-NOTES-2024.WEB.APP



SCAN ME TO GET THE WEBSITE

**YOU CAN ALSO GET TO THE WEBSITE BY TYPING
KTUNOTES.LIVE**



SCAN ME TO GET IN THE WHATSAPP GROUP

follow me on whatsapp



Welcome to a community built for students, by students. Unlock our extensive library of high-quality, premium notes, all available as PDFs. Whatever your subject, we have a variety of notes to help you excel. Have a question? Mail us your enquiries at ktunotes4@gmail.com

MODEL QUESTION PAPER				
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY				
THIRD SEMESTER B. TECH DEGREE EXAMINATION, MONTH AND YEAR				
Course Code: PBCST304				
Course Name: Object Oriented Programming				
Max. Marks: 40			Duration: 2 hours 30 minutes	
	PART A			
		Answer all questions. Each question carries 2 marks	CO	Marks
1		You compile MyApp.java on Windows into MyApp.class, then copy that same MyApp.class file to both a Linux machine and a macOS system. Without recompiling, you run: <pre>java MyApp</pre> on each machine, and it executes successfully. What is the name of the compiled file format that makes this possible?	CO1	(2)
2		What will be the value of <i>a</i> after the execution of the following code segment in Java. <pre>int a = -1; a = a >> 4;</pre>	CO1	(2)
3		You're given this Java code: <pre>class Vehicle { protected String type = "Generic Vehicle"; Vehicle() { System.out.println("Vehicle created"); } } class Car extends Vehicle { protected String type = "Car"; Car() { // **INSERT STATEMENT A HERE** System.out.println("Type: " + type); } void printParentType() { System.out.println("Parent Type: " +</pre>	CO2	(2)

	<pre> super.type); } } public class TestSuper { public static void main(String[] args) { Car c = new Car(); c.printParentType(); } } </pre> a. What exact statement must replace <code>// **INSERT STATEMENT A HERE**</code> so that when you run TestSuper, you see: <pre> Vehicle created Type: Car Parent Type: Generic Vehicle </pre> b. In one sentence each, explain: (i) What the statement you inserted does here. (ii) Why super.type prints "Generic Vehicle" while type prints "Car".		
4	Illustrate inheritance in Java.	CO2	(2)
5	Using proper examples, give the difference between interfaces and abstract classes in Java.	CO3	(2)
6	You're given this Java class: <pre> 1 import java.io.*; 2 public class ExceptionDemo { 3 public static int divide(int a, int b) { 4 return a / b; 5 } 6 public static void readFile(String path) throws IOException { 7 FileInputStream fis = new FileInputStream(path); 8 fis.close(); 9 } 10 public static void main(String[] args) { 11 String filePath = args[0]; 12 int x = Integer.parseInt(args[1]); 13 int y = Integer.parseInt(args[2]); </pre>	CO2	(2)

		<pre> 14 try { 15 readFile(filePath); 16 System.out.println("Result: " + divide(x, 17 y)); 18 } catch (Exception e) { 19 e.printStackTrace(); 20 } 21 } </pre> (i) At which lines in this code might a checked exception be thrown? (ii) At which lines might an unchecked exception occur? (iii) For each case, name the exception class and explain briefly why it's checked or unchecked.		
7		Give the differences between AWT and Swing in Java.	CO5	(2)
8		List the main steps to establish a JDBC connection. What is a PreparedStatement, and why is it preferred over a Statement?	CO5	(2)
PART B				
Answer any one full question from each module. Each question carries 6 marks				
Module 1				
9	a)	Write a Java program that uses command line arguments to calculate area: ● If the user enters one argument, the program should treat it as the side of a square and print the area of the square. ● If the user enters two arguments, the program should treat them as the length and breadth of a rectangle and print the area of the rectangle. ● If the number of arguments is not 1 or 2, display an appropriate error message.	CO1	(4)
	b)	Illustrate the use of <i>this</i> keyword in Java.	CO2	(2)
10	a)	Design a Java class Book with the following attributes: title, author, and price. ● Write a parameterized constructor to initialize these values. ● Also write a default constructor that sets default values. ● Create a display() method to print the book details. ● In the main method, create two objects — one using the default constructor and another using the parameterized constructor — and display their details.	CO2	(4)

	b)	Refactor the Book class so that (i) data validation and (ii) input/output each live in their own classes. Which SOLID principle are you applying, and why?	CO1	(2)
Module 2				
11	a)	Create a Java class Calculator that demonstrates method overloading by implementing the add() method in the following ways: ● add(int a, int b) — returns the sum of two integers. ● add(double a, double b) — returns the sum of two double values. ● add(int a, int b, int c) — returns the sum of three integers. In the main method, create an object of Calculator and call all the overloaded add() methods with appropriate arguments. Display the results.	CO2	(3)
	b)	Create a Java class Counter that has a static variable count to keep track of the number of objects created. Increment count in the constructor. In the main method, create three objects of the Counter class and display the total number of objects created using the static variable.	CO2	(3)
12		Define a Java base class Animal with a method makeSound() that prints a generic sound. Then, create two subclasses Dog and Cat that override the makeSound() method to print "Bark" and "Meow" respectively. In the main method, create objects of Dog and Cat and call the makeSound() method to show runtime polymorphism.	CO2	(3)
Module 3				
13		You have a third-party class AnalyticsService with method sendEvent(String). Write an Adapter so that it conforms to your Logger interface (log(String)). Show both class definitions and the adapter code.	CO4	(6)
14		Implement the Singleton pattern for a Logger class that writes log entries to a file. Show your code and explain how it ensures a single instance.	CO2	(6)
Module 4				
15		Design a simple Java Swing GUI application with the following features: ● A window with two buttons labeled "Greet" and "Clear". ● A text field where the user can enter their name. ● When the "Greet" button is clicked, display "Hello, [Name]!" in a label. ● When the "Clear" button is clicked, clear the text field and the label. Use appropriate event handling by implementing ActionListener and demonstrate how to register and handle events in Java Swing. In your Swing GUI, the user enters a name and clicks 'Save'. Write the	CO5	(6)

		JDBC code to insert that name into table greetings(name VARCHAR), handling exceptions appropriately. Write the complete code for this application.		
16	a)	Draw an MVC diagram for this Swing+JDBC app.	CO5	(3)
	b)	Describe how you'd apply Dependency Inversion so the Controller can be unit-tested without a real database.	CO5	(3)



APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY
THIRD SEMESTER B. TECH DEGREE EXAMINATION, MONTH AND YEAR
MODEL QUESTION PAPER

Course Code: PBCST304

Course Name: Object Oriented Programming

1. You compile MyApp.java on Windows into MyApp.class, then copy that same MyApp.class file to both a Linux machine and a macOS system. Without recompiling, you run:

java MyApp

on each machine, and it executes successfully. What is the name of the compiled file format that makes this possible?

Answer:

The compiled file format that allows you to run the same MyApp.class file on Windows, Linux, and macOS without recompiling is called **Java bytecode**, and it is specifically stored in a **.class file**.

Java bytecode is a platform-independent, intermediate representation that the Java Virtual Machine (JVM) can execute on any operating system with a compatible JVM, making Java programs highly portable.

2. What will be the value of *a* after the execution of the following code segment in Java.

```
int a = -1;
a = a >> 4;
```

Answer:

- In Java, the signed right shift operator (>>) fills the higher-order bits with the sign bit (1 for negative values).
- The binary representation of -1 is all 1's in two's complement (for a 32-bit int, it's 32 ones).
- Right shifting by any number of bits for -1 keeps all bits as 1 (since the sign bit is 1), so the value remains -1
- Therefore, after `a = a >> 4;`, *a* is still -1

3. You're given this Java code:

```
class Vehicle
{
    protected String type = "Generic Vehicle";
    Vehicle()
    {
        System.out.println("Vehicle created");
    }
}
class Car extends Vehicle
{
    protected String type = "Car";
    Car()
```

```

    {
        // **INSERT STATEMENT A HERE**
        System.out.println("Type: " + type);
    }
    void printParentType()
    {
        System.out.println("Parent Type: " + super.type);
    }
}
public class TestSuper
{
    public static void main(String[] args)
    {
        Car c = new Car();
        c.printParentType();
    }
}

```

- a. What exact statement must replace // **INSERT STATEMENT A HERE** so that when you run TestSuper, you see:
 Vehicle created
 Type: Car
 Parent Type: Generic Vehicle
- b. In one sentence each, explain:
 - (i) What the statement you inserted does here.
 - (ii) Why super.type prints "Generic Vehicle" while type prints "Car".

Answer:

- a. No need to add any statement there
 The constructor of Car class implicitly executes **super()**. It will call the constructor of its base class.
 - b.
 - i. The constructor of Car class implicitly executes **super()**. It will call the constructor of its base class.
 - ii. super.type in Car class refers to the variable in its parent class. So it will print "Generic Vehicle".
4. Illustrate inheritance in Java.

Answer:

- The mechanism of deriving a new class from an old class is called Inheritance.
- The old class is known as the **base class(or super class or parent class)**, and the new one is called the **subclass (or derived class or child class)**.
- A subclass is a specialized version of a superclass. It inherits all possible instance variables and methods defined by the superclass and adds its own, unique elements.
- To inherit a class, we should use the keyword **extends**.

- General form: `class subclass-name extends superclass-name`

```

{
    // body of class
}

```
- Example: We have a generic Animal class, and we want to create a more specific Dog class that inherits from it.

```

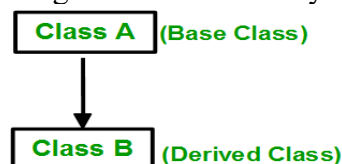
// Superclass
class Animal
{
    void eat()
    {
        System.out.println("This animal eats food.");
    }
}

// Subclass
class Dog extends Animal
{
    void bark()
    {
        System.out.println("The dog barks.");
    }
}

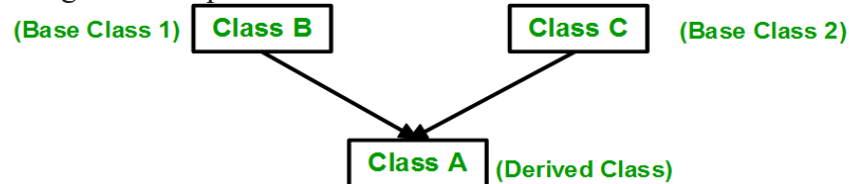
// Main class to test inheritance
public class TestInheritance
{
    public static void main(String[] args)
    {
        Dog myDog = new Dog();
        myDog.eat();    // Inherited method
        myDog.bark();   // Subclass method
    }
}

```

- Example: We have a generic Animal class, and we want to create a more specific Dog class that inherits from it.
- Inheritance may take several forms:
 1. Single Inheritance: Only one super class and one derived class



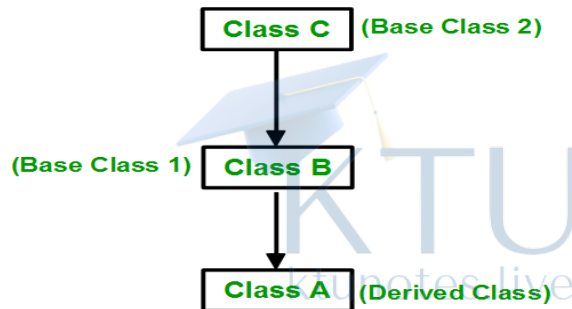
2. Multiple Inheritance: Several super classes for a single derived class. Java does not directly support this type of inheritance. This inheritance is implemented by using the concept of interfaces.



3. Hierarchical Inheritance: one super class, many subclasses



4. Multilevel Inheritance: a subclass becomes a superclass of another subclass



5. Using proper examples, give the difference between interfaces and abstract classes in Java.

Answer:

Interface	Abstract class
An interface contains methods and variables. These methods should be abstract, and variables should be static and final.	Any class that contains one or more abstract methods is called abstract class. Can have both abstract and concrete methods.
All methods are implicitly abstract	Must be explicitly declared with abstract keyword.
A class can implement multiple interfaces. Classes use the implements keyword to inherit interfaces Can extend other interfaces. Eg: interface B extends A <pre> { } </pre>	A class can extend only one abstract class. Classes use the extends keyword to inherit an abstract class Subclass must override all abstract methods
No constructors allowed.	Can have constructors (used by subclasses).

Interfaces are not used to instantiate objects directly.	Abstract classes are not used to instantiate objects directly.
Syntax: <pre> abstract class ClassName { abstract void method1(); void method2() { // Code } } </pre>	Syntax: <pre> interface InterfaceName { datatype const1=value; datatype const2=value; return_type method1(); return_type method2(); } </pre>
Example: <pre> interface Animal { void makeSound(); } class Dog implements Animal { public void makeSound() { System.out.println("Bark"); } } public class Main { public static void main(String[] args) { Dog obj = new Dog(); obj.makeSound(); } } </pre>	Example: <pre> abstract class Animal { abstract void makeSound(); void sleep() { System.out.println("Sleeping"); } } class Dog extends Animal { void makeSound() { System.out.println("Bark"); } } public class Main { public static void main(String[] args) { Dog obj = new Dog(); obj.makeSound(); obj.sleep(); } } </pre>

6. You're given this Java class:

```

1 import java.io.*;
2 public class ExceptionDemo {
3     public static int divide(int a, int b) {
4         return a / b;
5     }
6     public static void readFile(String path) throws IOException {
7         FileInputStream fis = new FileInputStream(path);
8         fis.close();
9     }

```

```

10 public static void main(String[] args) {
11     String filePath = args[0];
12     int x = Integer.parseInt(args[1]);
13     int y = Integer.parseInt(args[2]);
14     try {
15         readFile(filePath);
16         System.out.println("Result: " + divide(x,y));
17     } catch (Exception e) {
18         e.printStackTrace();
19     }
20 }
21 }

```

- (i) At which lines in this code might a checked exception be thrown?
- (ii) At which lines might an unchecked exception occur?
- (iii) For each case, name the exception class and explain briefly why it's checked or unchecked.

Answer:

- i. The line numbers 7 and 8 may throw checked exceptions
- ii. The line numbers 11, 12, 13 and 16 may throw checked exceptions.
- iii. **Checked Exceptions**

FileNotFoundException and IOException are subclasses of Exception (but **not** of RuntimeException). Checked exceptions are used for conditions outside the programmer's control, like missing files or I/O problems, so the compiler requires us to handle them explicitly (using try-catch or declaring them with throws)

The line 7 can throw a checked exception—specifically, FileNotFoundException, which is a subclass of IOException. It occurs when the file at the specified path does not exist or cannot be opened

The line 8 can also throw a checked exception—IOException. It occurs when an error occurs while closing the file input stream.

Both of these checked exceptions are handled by declaring throws IOException in the readFile method signature.

Unchecked Exceptions

```
String filePath = args[0];
```

This line can throw **NumberFormatException** if the argument is not a valid integer.

```
int x = Integer.parseInt(args[1]);
int y = Integer.parseInt(args[2]);
```

These lines can throw **ArrayIndexOutOfBoundsException** if fewer than three arguments are passed.

```
System.out.println("Result: " + divide(x,y));
```

This line can throw **ArithmeticException** if y is zero (division by zero in the divide method).

These exceptions are subclasses of RuntimeException. It occurs due to a programming error, so it does not need to be checked at compile time. So they are called unchecked exceptions. It is not required to handle unchecked exceptions explicitly with try-catch or throws.

7. Give the differences between AWT and Swing in Java.

Answer:

Feature	AWT (Abstract Window Toolkit)	Swing (Java Foundation Classes)
Introduction	Introduced in Java 1.0	Introduced in Java 1.2 as part of Java Foundation Classes (JFC)
Component Type	Heavyweight	Lightweight
Platform Dependency	Platform Dependent (appearance depends on OS)	Platform Independent (same look and feel on all OS)
Look and Feel	Native (appearance changes with the OS)	Can have pluggable look and feel (e.g., Metal, Nimbus, etc.)
Number of Components	Limited GUI components	Rich set of components (e.g., JTable, JTree, JTabbedPane, etc.)
Event Handling	Uses Delegation Event Model	Uses Delegation Event Model
Architecture	Does not follow a clear MVC pattern.	Follows a Model-View-Controller (MVC)-like architecture.
Performance	Faster for very simple UIs	Can be slightly slower but more feature-rich
Package	java.awt	javax.swing

8. List the main steps to establish a JDBC connection.
What is a PreparedStatement, and why is it preferred over a Statement?

Answer:

Steps to Create JDBC Application

o **Six steps :**

1. Import the packages:

use the following package : *import java.sql.**

2. Register the JDBC driver: To open a communication channel with the database.

`Class.forName("com.mysql.jdbc.Driver");`

Here "com.mysql.jdbc.Driver" is the driver class for mysql database.

3. Open a connection:

`Connection connect = DriverManager.getConnection
("jdbc:mysql://localhost:3306/VJCET","root","pass");`

jdbc: API
mysql: database management system
localhost: server name on which mysql is running
3306: port number
VJCET: database name
root: default user name of mysql database
pass: password given by the user at the time of installing mysql database

4. **Execute a query:** build and submit an SQL statement to the database.
PreparedStatement pstmt=connect.prepareStatement("SELECT id, first, last, age FROM Employees");
ResultSet rs = pstmt.executeQuery();
5. **Extract data from result set:** Use appropriate *ResultSet.getXXX()* method to retrieve the data from the result set
int id = rs.getInt("id"); //Retrieve by column name
String first = rs.getString(3); //Retrieve by column number
6. **Clean up the environment:** Requires explicitly closing all database resources versus relying on the JVM's garbage collection.
connect.close();

A **PreparedStatement** is a precompiled SQL statement used in Java to execute parameterized database queries. It allows us to write SQL queries with placeholders (?) for parameters that are set at runtime. The SQL query is sent to the database where it is precompiled and stored, and then can be executed multiple times with different parameter values

It is preferred over a regular **Statement** for several important reasons related to security, performance, functionality, code readability, and Maintainability.

PART B

9.
 - a) Write a Java program that uses command line arguments to calculate area:
 - If the user enters one argument, the program should treat it as the side of a square and print the area of the square.
 - If the user enters two arguments, the program should treat them as the length and breadth of a rectangle and print the area of the rectangle.
 - If the number of arguments is not 1 or 2, display an appropriate error message.

Answer:

```
public class ABC
{
    public static void main(String[] args)
    {
        int l,b;
        if(args.length == 1)
```

```

    {
        l = Integer.parseInt(args[0]);
        System.out.println("Area of the square: " + (l*l));
    }
    else if(args.length == 2)
    {
        l = Integer.parseInt(args[0]);
        b = Integer.parseInt(args[1]);
        System.out.println("Area of the square: " + (l*b));
    }
    else
    {
        System.out.println("Invalid number of arguments");
    }
}
}

```

b) Illustrate the use of *this* keyword in Java.

Answer:

In Java, "this" is a reference variable that refers to the current object instance. It is mainly used to,

- Call current class methods and fields
- To pass an instance of the current class as a parameter
- To differentiate between the local and instance variables.

Using "this" reference can improve code readability and reduce naming conflicts.

Different way to use "this" in Java

The following are the ways to use the "this" keyword in Java:

1. Using "this" keyword to refer to current class instance variables.
2. Using this() to invoke the current class constructor
3. Using 'this' keyword to return the current class instance
4. Using 'this' keyword as the method parameter
5. Using 'this' keyword to invoke the current class method
6. Using 'this' keyword as an argument in the constructor call

1. Using "this" to Refer to Current Class Instance Variables

```

class This
{
    int a, b;
    This(int a, int b)    // Parameterized constructor
    {
        this.a = a;
        this.b = b;
    }
    void display()
    {
        System.out.println("a = " + a + " b = " + b);
    }
    public static void main(String[] args)

```

```

        {
            This object = new This(10, 20);
            object.display();
        }
    }

```

Output: a = 10 b = 20

2. Using this() to Invoke Current Class Constructor

```

class This
{
    int a; int b;
    This() // Default constructor
    {
        this(10, 20);
        System.out.println("Inside default constructor \n");
    }
    This(int a, int b) // Parameterized constructor
    {
        this.a = a;
        this.b = b;
        System.out.println("Inside parameterized constructor");
    }
    public static void main(String[] args)
    {
        This object = new This();
    }
}

```

Output: Inside parameterized constructor
Inside default constructor

3. Using "this" Keyword to Return the Current Class Instance

```

class This
{
    int a, b;
    This() // Default constructor
    {
        a = 10;
        b = 20;
    }
    This get() // Method that returns current class instance
    {
        return this;
    }
    void display()
    {
        System.out.println("a = " + a + " b = " + b);
    }
    public static void main(String[] args)
    {
        This object = new This();
        object.get().display();
    }
}

```

Output: a = 10 b = 20

4. Using "this" Keyword as a Method Parameter


```

class This
{
    int a; int b;
    This() // Default constructor
    {
        a = 10;
        b = 20;
    }
    void display(This obj) // Method that receives "this" keyword as parameter
    {
        System.out.println("a = " + obj.a+ " b = " + obj.b);
    }
    void get() // Method that returns current class instance
    {
        display(this);
    }
    public static void main(String[] args)
    {
        This object = new This();
        object.get();
    }
}

```

Output: a = 10 b = 20

5. Using "this" Keyword to Invoke the Current Class Method

```

class This
{
    void display()
    {
        this.show(); // calling function show()
        System.out.println("Inside display function");
    }
    void show()
    {
        System.out.println("Inside show function");
    }
    public static void main(String args[])
    {
        This g1 = new This();
        g1.display();
    }
}

```

Output: Inside show function
Inside display function

6. Using "this" Keyword as an Argument in the Constructor Call

```

class A
{
    B obj;
    A(B obj) // Parameterized constructor with object of B as a
    parameter
    {
        this.obj = obj;
        obj.display();
    }
}
class B

```

```

{
    int x = 5;
    // Default Constructor that create an object of A with passing this as an
    // argument //in the constructor
    B()
    {
        A obj = new A(this);
    }
    void display()
    {
        System.out.println("Value of x in Class B : " + x);
    }
    public static void main(String[] args)
    {
        B obj = new B();
    }
}

```

Output: Value of x in Class B : 5

Advantages of Using "this" Reference

- It helps to distinguish between instance variables and local variables with the same name.
- It can be used to pass the current object as an argument to another method.
- It can be used to return the current object from a method.
- It can be used to invoke a constructor from another overloaded constructor in the same class.

Disadvantages of Using "this" Reference

- Overuse of this can make the code harder to read and understand.
- Using this unnecessarily can add unnecessary overhead to the program.
- Using this in a static context results in a compile-time error.
- Overall, this keyword is a useful tool for working with objects in Java, but it should be used judiciously and only when necessary

10.

- a) Design a Java class Book with the following attributes: title, author, and price.
- Write a parameterized constructor to initialize these values.
 - Also write a default constructor that sets default values.
 - Create a display() method to print the book details.
 - In the main method, create two objects — one using the default constructor and another using the parameterized constructor — and display their details.

Answer:

```
class Book
```

```
{
    String title;
    String author;
    double price;
```

```
    // Default constructor
```

```

Book()
{
    title = "Unknown";
    author = "Unknown";
    price = 0.0;
}

// Parameterized constructor
Book(String title, String author, double price)
{
    this.title = title;
    this.author = author;
    this.price = price;
}

void display()
{
    System.out.println("Title: " + title);
    System.out.println("Author: " + author);
    System.out.println("Price: " + price);
}
}

public class Sample
{
    public static void main(String[] args)
    {
        Book book1 = new Book(); // Using default constructor
        // Using parameterized constructor
        Book book2 = new Book("Hello Java", "George Orwell", 999.99);

        System.out.println("Default Book:");
        book1.display();

        System.out.println("\nParameterized Book:");
        book2.display();
    }
}

```

- b) Refactor the Book class so that (i) data validation and (ii) input/output each live in their own classes. Which SOLID principle are you applying, and why?

Answer:

Refactoring the Book class by separating data validation and input/output into their own classes is an application of the **Single Responsibility Principle (SRP)** from the SOLID design principles. SRP dictates that each class should have only one reason to change, meaning only one responsibility. By splitting validation and I/O from the core model, each class is easier to maintain and extend.

The original Book class likely handles multiple concerns:

- Storing book data
- Validating that data
- Reading from or writing to input/output sources

By refactoring, you're separating these concerns into distinct classes:

- Book: Holds the book's data (title, author, etc.)
- BookValidator: Ensures the data is valid
- BookIO: Handles input/output operations

This makes each class easier to maintain, test, and extend independently.

Refactored Structure Example

// Book.java: Just holds data

```
class Book
{
    String title;
    String author;
    double price;

    Book(String title, String author, double price)
    {
        this.title = title;
        this.author = author;
        this.price = price;
    }
}
```

// BookValidator.java: Handles validation

```
class BookValidator
{
    boolean isValid(Book book)
    {
        return (book.title != null && !book.title.isEmpty() && book.author != null
            && !book.author.isEmpty() && book.price >= 0);
    }
}
```

// BookIO.java: Handles input/output

```
class BookIO
{
    void display(Book book)
    {
        System.out.println("Title: " + book.title);
        System.out.println("Author: " + book.author);
        System.out.println("Price: " + book.price);
    }
}
```

11.

a) Create a Java class Calculator that demonstrates method overloading by implementing the add() method in the following ways:

- add(int a, int b) — returns the sum of two integers.
- add(double a, double b) — returns the sum of two double values.
- add(int a, int b, int c) — returns the sum of three integers.

In the main method, create an object of Calculator and call all the overloaded add() methods with appropriate arguments. Display the results.

Answer:

```
class Calculator
```

```
{  
    int add(int a, int b)           // Adds two integers  
    {  
        return a + b;  
    }  
  
    double add(double a, double b) // Adds two double values  
    {  
        return a + b;  
    }  
  
    int add(int a, int b, int c)    // Adds three integers  
    {  
        return a + b + c;  
    }  
}
```

```
public class Sample
```

```
{  
    public static void main(String[] args)  
    {  
        Calculator calc = new Calculator();  
  
        int sum1 = calc.add(5, 7);           // Calls add(int, int)  
        double sum2 = calc.add(4.5, 8.3);    // Calls add(double, double)  
        int sum3 = calc.add(2, 3, 4);        // Calls add(int, int, int)  
  
        System.out.println("Sum of two ints: " + sum1);  
        System.out.println("Sum of two doubles: " + sum2);  
        System.out.println("Sum of three ints: " + sum3);  
    }  
}
```

b) Create a Java class Counter that has a static variable count to keep track of the number of objects created. Increment count in the constructor. In the main method, create three objects of the Counter class and display the total number of objects created using the static variable.

Answer:

```
class Counter
{
    static int count = 0;
    public Counter() // Constructor increments static count
    {
        count++;
    }

    // Static method to get the count
    public static int getCount()
    {
        return count;
    }
}

public class Sample
{
    public static void main(String[] args)
    {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();

        System.out.println("Total objects created: " + Counter.getCount());
    }
}
```

12. Define a Java base class Animal with a method makeSound() that prints a generic sound. Then, create two subclasses Dog and Cat that override the makeSound() method to print "Bark" and "Meow" respectively. In the main method, create objects of Dog and Cat and call the makeSound() method to show runtime polymorphism.

Answer:

```
class Animal
{
    void makeSound()
    {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal
{
    void makeSound()
    {
        System.out.println("Bark");
    }
}
```

```

}
class Cat extends Animal
{
    void makeSound()
    {
        System.out.println("Meow");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Animal a;
        Dog d = new Dog();
        Cat c = new Cat();
        a = d;
        a.makeSound(); // Output: Bark
        a = c;
        a.makeSound(); // Output: Meow
    }
}

```

13. You have a third-party class `AnalyticsService` with method `sendEvent(String)`. Write an Adapter so that it conforms to your `Logger` interface (`log(String)`). Show both class definitions and the adapter code.

Answer:

```

public interface Logger
{
    void log(String message);
}
public class AnalyticsService
{
    public void sendEvent(String event)
    {
        // Sends event to analytics backend
        System.out.println("Analytics event sent: " + event);
    }
}
public class AnalyticsLoggerAdapter implements Logger
{
    private AnalyticsService analyticsService;

    public AnalyticsLoggerAdapter(AnalyticsService analyticsService)
    {
        this.analyticsService = analyticsService;
    }
}

```

```

    }

    public void log(String message)
    {
        analyticsService.sendEvent(message);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        AnalyticsService analyticsService = new AnalyticsService();
        Logger logger = new AnalyticsLoggerAdapter(analyticsService);

        logger.log("User clicked 'Buy Now'");
    }
}

```

Adapter Pattern bridges the gap between Logger and AnalyticsService. You can now use AnalyticsService wherever a Logger is expected — without changing either class. This promotes Open/Closed Principle (OCP) from SOLID: classes are open for extension but closed for modification.

14. Implement the Singleton pattern for a Logger class that writes log entries to a file. Show your code and explain how it ensures a single instance.

(File- out of syllabus)

Answer:

```

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Logger
{
    private static Logger instance;
    private BufferedWriter writer;

    // Private constructor prevents instantiation from outside
    private Logger()
    {
        try
        {
            writer = new BufferedWriter(new FileWriter("log.txt", true)); // Append mode
        }
        catch (IOException e)
        {
        }
    }
}

```



```

        e.printStackTrace();
    }
}

// Public method to provide access to the single instance
public static Logger getInstance()
{
    if (instance == null)
    {
        synchronized (Logger.class)
        {
            if (instance == null)
            {
                instance = new Logger();
            }
        }
    }
    return instance;
}

// Method to write a log entry
public void log(String message)
{
    try
    {
        writer.write(message);
        writer.newLine();
        writer.flush();
    }
    catch (IOException e)
    {
        e.printStackTrace();
    }
}

// Optional: close the writer when done
public void close()
{
    try
    {
        if (writer != null) writer.close();
    }
    catch (IOException e)
    {
        e.printStackTrace();
    }
}

```

```
}
}
```

- Private constructor: Prevents external classes from creating new instances using new Logger().
- Static getInstance() method: Provides a global access point to the single instance.
- Double-checked locking: Ensures thread safety while avoiding unnecessary synchronization after the instance is initialized.
- Static field instance: Holds the single reference to the Logger object.
This guarantees that only one Logger object exists throughout the application, and all log entries go to the same file via that instance.

15. Design a simple Java Swing GUI application with the following features:

- A window with two buttons labeled “Greet” and “Clear”.
- A text field where the user can enter their name.
- When the “Greet” button is clicked, display “Hello, [Name]!” in a label.
- When the “Clear” button is clicked, clear the text field and the label.

Use appropriate event handling by implementing ActionListener and demonstrate how to register and handle events in Java Swing. In your Swing GUI, the user enters a name and clicks ‘Save’. Write the JDBC code to insert that name into table greetings(name VARCHAR), handling exceptions appropriately. Write the complete code for this application.

Answer:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
```

```
class GreetingApp implements ActionListener
{
    JFrame f;
    JTextField nameField;
    JLabel greetingLabel;
    JButton greetButton, clearButton, saveButton;
```

```
public GreetingApp()
{
    f = new JFrame();
    f.setTitle("Greeting App");
    f.setSize(400, 200);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setLayout(new FlowLayout());

    nameField = new JTextField(20);
    greetingLabel = new JLabel("");
    greetButton = new JButton("Greet");
```

```

clearButton = new JButton("Clear");
saveButton = new JButton("Save");

f.add(new JLabel("Enter your name:"));
f.add(nameField);
f.add(greetButton);
f.add(clearButton);
f.add(saveButton);
f.add(greetingLabel);

// Register event listeners
greetButton.addActionListener(this);
clearButton.addActionListener(this);
saveButton.addActionListener(this);

f.setVisible(true);
}

public void actionPerformed(ActionEvent e)
{
    String name = nameField.getText().trim();

    if (e.getSource() == greetButton)
    {
        if (!name.isEmpty())
        {
            greetingLabel.setText("Hello, " + name + "!");
        }
        else
        {
            greetingLabel.setText("Please enter your name.");
        }
    }
    else if (e.getSource() == clearButton)
    {
        nameField.setText("");
        greetingLabel.setText("");
    }
    else if (e.getSource() == saveButton)
    {
        if (!name.isEmpty())
        {
            saveNameToDatabase(name);
        }
        else
        {

```

```

        JOptionPane.showMessageDialog(this, "Name field is empty!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

private void saveNameToDatabase(String name)
{
    String url = "jdbc:mysql://localhost:3306/your_database";
    String user = "uname";
    String password = "pass";

    String sql = "INSERT INTO greetings(name) VALUES(?)";

    try
    {
        Connection conn = DriverManager.getConnection(url, user, password);
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString(1, name);
        int rowsInserted = stmt.executeUpdate();

        if (rowsInserted > 0)
        {
            JOptionPane.showMessageDialog(this, "Name saved successfully!");
        }

        stmt.close();
        conn.close();
    }
    catch (SQLException ex)
    {
        JOptionPane.showMessageDialog(this, "Database error: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
        ex.printStackTrace();
    }
}

public static void main(String[] args)
{
    try
    {
        Class.forName("com.mysql.jdbc.Driver"); // Load MySQL JDBC driver
    }
    catch (ClassNotFoundException e)
    {
        System.out.println("MySQL JDBC Driver not found.");
    }
}

```

```

        GreetingApp obj = new GreetingApp();
    }
}

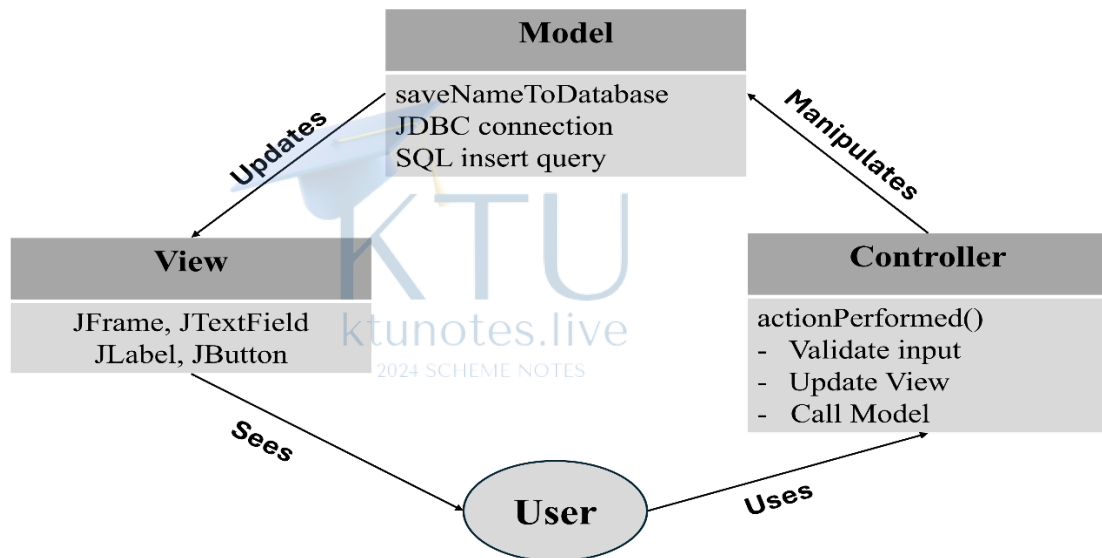
```

16.

a) Draw an MVC diagram for this Swing + JDBC app.

Answer:

MVC Component	Code Element(s)	Responsibility
Model	saveNameToDatabase()	Interacts with MySQL database
View	JFrame, JTextField, JLabel, JButton, JOptionPane	Displays UI and results
Controller	actionPerformed()	Handles button clicks, validation, and logic



b) Describe how you'd apply Dependency Inversion so the Controller can be unit-tested without a real database.

Answer:

Dependency Inversion means:

- High-level modules (Controller) should not depend on low-level modules (JDBC code).
- Both should depend on abstractions
- Abstractions should not depend on details; details should depend on abstractions.

To apply the Dependency Inversion Principle (DIP) in your Swing + JDBC application and make the Controller unit-testable without a real database, follow these steps:

1. Introduce an Abstraction (Interface)

Instead of letting the Controller directly depend on JDBC code (a low-level module), define an interface that represents the behavior needed.

Eg:

```
public interface GreetingRepository
{
    boolean saveName(String name);
}
```

This interface acts as an abstraction that both the Controller and the database logic will depend on.

2. Make the Controller Depend on the Interface

Modify the Controller so it receives a GreetingRepository object (not a concrete JDBC class). This is typically done via constructor injection:

```
public class GreetingController
{
    private GreetingRepository repository;

    public GreetingController(GreetingRepository repository)
    {
        this.repository = repository;
    }

    public void handleSave(String name)
    {
        repository.saveName(name);
    }
}
```

Now the Controller is decoupled from the database — it only knows about the interface.

3. Use a Mock Implementation for Testing

During unit testing, you can pass a mock or fake implementation of GreetingRepository that simulates database behavior:

```
public class MockGreetingRepository implements GreetingRepository
{
    public boolean saveName(String name)
    {
        // Simulate success without touching a real database
        return true;
    }
}
```

This allows you to test the Controller's logic in isolation, without needing a real database connection.

By applying Dependency Inversion:

- The Controller (high-level module) depends on an interface, not a concrete JDBC class.
- The JDBC logic (low-level module) also implements that interface.
- This makes the Controller easily testable, flexible, and loosely coupled — a core benefit of clean architecture.





KTUNOTES **4**

WhatsApp group



[https://chat.whatsapp.com/](https://chat.whatsapp.com/CUJvxbowcKt9zARITH0vH1)
[CUJvxbowcKt9zARITH0vH1](https://chat.whatsapp.com/CUJvxbowcKt9zARITH0vH1)



KTUNOTES.LIVE

KTU 2024 SCHEME NOTES

100% FREE RESOURCES

YOU CAN ACCESS B.TECH NOTES ON OUR THREE WEBSITES.

KTU2024.WEB.APP



SCAN ME TO GET THE WEBSITE

KTUNOTES.LIVE



SCAN ME TO GET THE WEBSITE

KTU-NOTES-2024.WEB.APP



SCAN ME TO GET THE WEBSITE

**YOU CAN ALSO GET TO THE WEBSITE BY TYPING
KTUNOTES.LIVE**



SCAN ME TO GET IN THE WHATSAPP GROUP

follow me on whatsapp



Welcome to a community built for students, by students. Unlock our extensive library of high-quality, premium notes, all available as PDFs. Whatever your subject, we have a variety of notes to help you excel. Have a question? Mail us your enquiries at ktunotes4@gmail.com